StudentHub

Complete Project Documentation

Student & Course Management Platform

TECHNOLOGY STACK

Backend: Spring Boot 3.2.5 | Java 21 LTS | Spring Security 6

Database: MySQL 8 | Hibernate 6 | Spring Data JPA 3.x

Frontend: React 18 | Vite 5 | React Router v7 | Axios 1.x

Auth: JWT (JJWT 0.11.5) | BCrypt Password Encoding

Build: Apache Maven 3.9.6 | npm | Vite Dev Server

Roles: ROLE_SUPER_ADMIN | ROLE_ADMIN | ROLE_STUDENT

Date: June 2026

Chapter 1: Project Overview & Introduction

StudentHub is a full-stack web application designed to streamline management of students, courses, and enrollments across educational institutions. Built with Spring Boot (backend) and React (frontend), it implements industry-standard patterns: Layered Architecture, RESTful API design, and JWT-based stateless authentication.

The system revolves around a three-role hierarchy — Super Admin, Institute Admin, and Student — each with precisely scoped data access and operations. This documentation covers every component from basics to advanced design decisions, explaining not just "what" the code does but "why" architectural choices were made.

1.1 Project Goals

- Centralized multi-tenant platform for managing educational data across multiple institutes
- Role-Based Access Control (RBAC) with strict data isolation between institutes
- Clean RESTful API consumed by a React SPA (Single Page Application)
- Demonstrate industry-grade patterns: DTO, Layered Architecture, Global Exception Handling, JWT
- Super Admin oversight of all institutes while keeping each institute's data fully isolated

1.2 Technology Stack

Technology	Version	Role
Spring Boot	3.2.5	Backend Framework — Auto-config, Embedded Tomcat, Starter POMs
Java	21 LTS	Programming Language — Strong typing, Streams API, OOP
Spring Security	6.x	Auth & Authorization — JWT Filter Chain, Method Security
Spring Data JPA	3.x	ORM Layer — Repository pattern, Derived query generation
Hibernate	6.x	JPA Implementation — Entity mapping, Schema management
MySQL	8.x	Relational Database — Tables, Foreign Keys, ACID Transactions
JJWT	0.11.5	JWT Library — Token generation & HMAC-SHA256 validation
Lombok	Latest	Code Generation — @Data, @Builder, @NoArgsConstructor
React	18.x	Frontend UI Library — Components, Hooks, Context API
Vite	5.x	Build Tool — Fast HMR, ESM-based dev server
Axios	1.x	HTTP Client — Request/Response interceptors, Promise-based
Maven	3.9.6	Build Tool — Dependency management, Build lifecycle

1.3 Why These Technologies?

Spring Boot — Auto-Configuration

Spring Boot eliminates boilerplate configuration through "auto-configuration". It scans the classpath, detects present libraries, and wires them with sensible defaults. The "Starter" dependency system (e.g., spring-boot-starter-web) bundles Spring MVC + Jackson JSON processor + Embedded Apache Tomcat into one dependency — a fully running HTTP server with zero XML configuration.

> **i INFO**
> spring-boot-starter-web single dependency gives you: (1) Spring MVC controller framework, (2) Jackson for automatic JSON serialization/deserialization, (3) Embedded Tomcat — no external server deployment needed, (4) Auto-configured DispatcherServlet. All from one Maven dependency.

JWT — Stateless Authentication

JWT is self-contained — the token carries the user's identity (email as subject claim) and is cryptographically signed. The server validates the signature on every request without any database lookup or session store. This makes the backend stateless and horizontally scalable. Any server instance can authenticate any request independently.

React + Vite

React's component model enables building complex UIs from isolated, reusable pieces. Vite uses native ES modules during development, making hot module replacement nearly instantaneous. React Router v7 handles client-side navigation without full page reloads — the SPA loads once and JavaScript swaps components based on URL.

Chapter 2: Project Structure & Directory Layout

StudentHub is a monorepo with clean separation between backend and frontend. Each directory has a single, well-defined purpose following the principle of high cohesion and low coupling.

2.1 Root Directory Layout

```
StudentHub/
% % %  backend/                          Spring Boot application (Java)
%    % % %  pom.xml                       Maven build descriptor — project identity +
dependencies
%    % % %  .classpath                    Eclipse JDT classpath (JRE container: JavaSE-21)
%    % % %  .settings/org.eclipse.jdt.core.prefs   Compiler compliance level (Java 21)
%    % % %  src/main/
%        % % %  java/com/studenthub/     All Java source code (see section 2.2)
%        % % %  resources/
%            % % %  application.properties   App config: DB URL, JPA settings, JWT config
%            % % %  schema.sql              Reference SQL DDL (Hibernate auto-creates tables)
% % %  frontend/                         React SPA (JavaScript)
%    % % %  package.json                  npm dependencies + build scripts
%    % % %  vite.config.js               Vite build + dev server configuration
%    % % %  src/                         All React source code (see section 2.3)
% % %  maven/apache-maven-3.9.6/        Bundled Maven binary (no system Maven required)
```

2.2 Backend Java Package Structure

Organized in packages that map directly to layers of the Layered Architecture. Each package has exactly one responsibility — Single Responsibility Principle at the package level.

```
com.studenthub/
% % %  StudentHubApplication.java     @SpringBootApplication entry point + CommandLineRunner
seed
% % %  controller/                    PRESENTATION LAYER — HTTP request/response handlers
%    % % %  AuthController.java        /api/auth/** — register, login, me, institutes
%    % % %  AdminController.java       /api/admin/** — institute admin operations
%    % % %  CourseController.java      /api/courses/** — paginated course CRUD
%    % % %  EnrollmentController.java  /api/enrollments/** — enrollment lifecycle
%    % % %  StudentController.java     /api/students/** — student profile management
%    % % %  SuperAdminController.java  /api/super-admin/** — platform-level control
% % %  service/                       BUSINESS LOGIC LAYER — all business rules live here
%    % % %  UserService.java           Auth, registration, cascading deletes, profile updates
%    % % %  CourseService.java         Course CRUD with institute scoping enforcement
%    % % %  StudentService.java        Student profile management + search
%    % % %  EnrollmentService.java     Enrollment lifecycle + business rule validation
%    % % %  DashboardService.java      Aggregated stats computation
% % %  repository/                    DATA ACCESS LAYER — Spring Data JPA interfaces
%    % % %  UserRepository.java        queries on users table
%    % % %  CourseRepository.java      queries with pagination + search on courses table
%    % % %  StudentRepository.java     queries on students table
%    % % %  EnrollmentRepository.java  queries + bulk deletes + counts on enrollments
% % %  entity/                        DATABASE TABLE MAPPINGS — JPA annotated classes
%    % % %  User.java                  users table (all accounts: admin/student/superadmin)
%    % % %  UserRole.java              Enum: ROLE_ADMIN | ROLE_STUDENT | ROLE_SUPER_ADMIN
%    % % %  Course.java                courses table (ManyToOne !’ users as institute)
%    % % %  Student.java               students table (OneToOne + ManyToOne !’ users)
%    % % %  Enrollment.java            enrollments table (bridge: student-course)
% % %  dto/                           DATA TRANSFER OBJECTS — API contract classes
%    % % %  RegisterRequest.java       registration payload + Bean Validation annotations
%    % % %  AuthRequest.java           login payload: {email, password}
%    % % %  AuthResponse.java          login response: {token, role, studentId, ...}
%    % % %  CourseDto.java             course data for API (no sensitive fields)
%    % % %  StudentDto.java            student data for API (no hashed password)
%    % % %  EnrollmentDto.java         enrollment data (flat: names instead of nested objects)
%    % % %  DashboardStatsDto.java     aggregated stats: totalStudents, totalCourses, etc.
%    % % %  InstituteDto.java          lightweight: {id, name, email} for dropdown lists
% % %  security/                      SECURITY INFRASTRUCTURE
%    % % %  SecurityConfig.java        Security filter chain: CORS, CSRF, session, auth rules
%    % % %  JwtTokenProvider.java      generateToken() on login, validateToken() on requests
%    % % %  JwtAuthenticationFilter.java  OncePerRequestFilter: extracts JWT from header
%    % % %  CustomUserDetailsService.java UserDetailsService impl: loads user by email from DB
% % %  exception/                     EXCEPTION HANDLING
    % % %  ResourceNotFoundException.java  throws 404 — entity not found
    % % %  BadRequestException.java        throws 400 — business rule violation
    % % %  GlobalExceptionHandler.java     @RestControllerAdvice: centralized JSON error
builder
```

2.3 Frontend Source Structure

```
src/
% % %  main.jsx             ReactDOM.createRoot — mounts <App /> into <div id="root"> in
index.html
% % %  App.jsx              BrowserRouter + all route definitions + AuthProvider wrapper
% % %  index.css            Global CSS variables, reset, shared utility classes (23KB)
% % %  components/
%    % % %  Navbar.jsx       Top navigation — role-aware links (different nav for admin/student)
%    % % %  Footer.jsx       Simple page footer
%    % % %  ProtectedRoute.jsx  Route guard: redirects if not authenticated or wrong role
% % %  context/
%    % % %  AuthContext.jsx  Global auth state: {user, login, logout, isAdmin, isStudent...}
% % %  services/
%    % % %  api.js           Axios instance: baseURL=localhost:8080/api, request/response
interceptors
% % %  pages/
    % % %  common/          Public pages (no authentication required)
    %    % % %  Home.jsx     Landing page with feature cards and call-to-action buttons
    %    % % %  Login.jsx    Login form: email/password !’ JWT storage !’ role-based redirect
    %    % % %  Register.jsx Registration form: tabs for Student vs Institute account types
    %    % % %  About.jsx    Static about page
    %    % % %  Contact.jsx  Contact information page
    % % %  admin/           Admin-role and Super Admin pages
    %    % % %  AdminDashboard.jsx      Institute admin panel: stats + students + courses +
enrollments
    %    % % %  AdminProfile.jsx        Edit institute profile (name, email, password)
    %    % % %  SuperAdminDashboard.jsx Super Admin platform panel: manage institutes + data
    %    % % %  SuperAdminProfile.jsx   Edit super admin credentials
    % % %  student/
        % % %  StudentDashboard.jsx    Student portal: courses, enrollment, profile editing
```

Chapter 3: Architecture & Design Patterns

StudentHub follows N-Tier Layered Architecture on the backend. Dependencies flow strictly downward: Controller depends on Service; Service depends on Repository; Repository depends on the database. No layer skips levels — a Controller never directly calls a Repository.

3.1 The Four Layers

Layer 1 — Presentation (Controllers)

Receive HTTP requests, extract data, delegate to Service, return HTTP responses. Never contain business logic. @RestController = @Controller + @ResponseBody: all return values auto-serialized to JSON by Jackson.

> **i INFO**
> Separation of Concerns: If you find SQL in a controller or HTTP parsing in a service, that is a code smell. Each layer should know only its own job. This makes each layer independently testable — you can mock the service layer to test the controller without a database.

Layer 2 — Business Logic (Services)

Contain all business rules, authorization checks, data transformation, and multi-step orchestration. @Service registers the class as a Spring bean. @Transactional wraps methods in database transactions via AOP proxy classes generated by Spring.

> **i INFO**
> @Transactional uses AOP (Aspect-Oriented Programming): Spring wraps your service class in a generated proxy. When you call a @Transactional method, the proxy starts a DB transaction, calls the real method, then commits on success or rolls back on any RuntimeException. Your business code stays clean — no explicit transaction.begin() / commit().

Layer 3 — Data Access (Repositories)

Interfaces extending JpaRepository. Spring auto-generates SQL implementations at startup by parsing method names. findByInstituteAndCourseNameContainingIgnoreCase generates: SELECT * FROM courses WHERE institute_id = ? AND LOWER(course_name) LIKE LOWER('%?%').

> **i INFO**
> Derived Query rules: findBy !' WHERE, And/Or !' logical operators, Containing !' LIKE %X%, IgnoreCase !' LOWER() wrapping, OrderBy !' ORDER BY clause, Count !' SELECT COUNT, Delete !' DELETE. Spring parses method names at startup and caches the generated SQL. Zero SQL written by the developer.

Layer 4 — Data (Entities + Database)

@Entity classes are POJOs that Hibernate maps to database tables. @Table specifies the table name, @Column configures columns, @ManyToOne/@OneToOne define foreign key relationships. Hibernate handles INSERT/UPDATE/SELECT/DELETE generation automatically.

3.2 DTO Pattern — Why It Matters

Using JPA entity objects directly in REST responses has critical problems:

- Security Risk: Entity.password (BCrypt hash) could accidentally appear in API responses

- Coupling Risk: Renaming a database column immediately breaks the API contract
- N+1 Problem: Lazy-loaded associations may trigger extra SQL queries during JSON serialization
  DTOs provide a clean, explicit contract between the API and clients, independent of the internal data model.

```
// Entity — internal representation (has sensitive fields)
public class Student {
    private User user;      // has user.password (BCrypt hash — NEVER expose!)
    private User institute; // has institute.password
    private String department;
}

// DTO — what the API exposes (safe, flat, client-optimized)
public class StudentDto {
    private String fullName;      // Flattened from student.user.fullName
    private String email;         // Flattened from student.user.email
    private String department;
    private String instituteName; // Flattened from student.institute.fullName
    // password: INTENTIONALLY ABSENT
    // The service layer does the mapping: Entity !’ DTO before returning to controller
}
```

3.3 RESTful API Principles

HTTP Method	Operation	Idempotent?	Typical Status Code
GET	Retrieve data (no side effects)	Yes	200 OK
POST	Create new resource	No	201 Created
PUT	Full update of existing resource	Yes	200 OK
DELETE	Remove resource	Yes	204 No Content

3.4 Request-Response Lifecycle

- 1. Client HTTP request !’ Embedded Tomcat (port 8080)
- 2. JwtAuthenticationFilter: read Authorization: Bearer {token} header
- 3. Validate JWT signature + expiry !’ extract email from sub claim
- 4. Load user from DB !’ populate SecurityContext with user + roles
- 5. Spring Security evaluates URL rules + @PreAuthorize annotations
- 6. @RestController method called with parsed request data
- 7. Service layer: business logic, validation, DB operations
- 8. Repository layer: SQL generated and executed against MySQL
- 9. Service maps Entity !’ DTO !’ Controller wraps in ResponseEntity<T>
- 10. Jackson serializes DTO to JSON !’ HTTP response sent
- 11. Any exception !’ GlobalExceptionHandler !’ structured JSON error response

Chapter 4: Entity Classes & Database Design

Four JPA entities map to four MySQL tables. All entity relationships use FetchType.EAGER — related objects are loaded in the same SQL query using JOINs to avoid the N+1 query problem.

4.1 Relationships Overview

- users: Central table. One record per user regardless of role (admin, student, superadmin)
- students: Student-specific data. OneToOne !' users (own account), ManyToOne !' users (institute)
- courses: Belongs to an institute. ManyToOne !' users where role=ROLE_ADMIN
- enrollments: Bridge table for Many-to-Many between students and courses. Adds status field

i INFO

Single Table for All Roles: One users table for Admin, Student, and SuperAdmin simplifies authentication — every login queries one table. Role-specific data lives in the students table (department, phone). This trades some normalization for operational simplicity and authentication efficiency.

4.2 User Entity

```
@Entity
@Table(name = "users")
@Data @NoArgsConstructor @AllArgsConstructor @Builder
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)  // AUTO_INCREMENT in MySQL
    private Long id;

    @Column(name = "full_name", nullable = false)  // NOT NULL, maps to column full_name
    private String fullName;

    @Column(nullable = false, unique = true)   // NOT NULL + UNIQUE constraint on email
    private String email;

    @Column(nullable = false)   // Stores BCrypt hash — 60 chars, never plaintext
    private String password;

    @Enumerated(EnumType.STRING)        // Stores "ROLE_ADMIN" as text, not ordinal
integer
    @Column(nullable = false)
    private UserRole role;              // ROLE_ADMIN | ROLE_STUDENT | ROLE_SUPER_ADMIN
}

// UserRole enum:
public enum UserRole {
    ROLE_ADMIN,          // Institute administrator
    ROLE_STUDENT,        // Enrolled student
    ROLE_SUPER_ADMIN     // Platform owner
}
```

i INFO

@Enumerated(EnumType.STRING) stores "ROLE_ADMIN" text. ORDINAL would store 0, 1, 2. If you reorder enum constants (e.g., insert a new role between existing ones), ORDINAL values become corrupt — all existing records would point to wrong roles. STRING is always safe.

4.3 Student Entity

```
@Entity
@Table(name = "students")
public class Student {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "student_id")
    private Long studentId;

    @OneToOne(cascade = CascadeType.ALL, fetch = FetchType.EAGER)
    @JoinColumn(name = "user_id", referencedColumnName = "id", nullable = false)
    private User user;
    // Each Student has exactly ONE User account
    // cascade=ALL: deleting Student also deletes the User record
    // fetch=EAGER: User loaded with Student in same SQL query (JOIN)
    // JoinColumn: students.user_id !’ users.id

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "institute_id", referencedColumnName = "id")
    private User institute;
    // Many Students belong to one Institute (a User with ROLE_ADMIN)
    // Self-referencing FK: both user_id and institute_id reference users.id
    // institute can be null if the institute is deleted (ON DELETE SET NULL in SQL)

    @Column(length = 15)
    private String phone;             // Optional, max 15 chars, validated as 10 digits

    @Column(nullable = false)
    private String department;        // Required at registration
}
```

4.4 Course Entity

```
@Entity
@Table(name = "courses")
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "course_id")
    private Long courseId;

    @Column(name = "course_name", nullable = false, length = 150)
    private String courseName;

    @Column(columnDefinition = "TEXT")   // TEXT type: no length limit, for long
descriptions
    private String description;

    @Column(length = 50)
    private String duration;            // e.g. "6 months", "40 hours"

    @Column(name = "instructor_name", length = 100)
    private String instructorName;

    @Column(columnDefinition = "TEXT")  // Course syllabus / material text
    private String content;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "institute_id", referencedColumnName = "id")
    private User institute;            // Which institute owns this course

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;   // updatable=false: JPA never includes in UPDATE
SQL

    @PrePersist                         // JPA lifecycle hook: runs before every INSERT
    protected void onCreate() {
        createdAt = LocalDateTime.now();  // Auto-set creation timestamp
    }
}
```

4.5 Enrollment Entity

```
@Entity
@Table(
    name = "enrollments",
    uniqueConstraints = {
        @UniqueConstraint(columnNames = {"student_id", "course_id"})
        // One student cannot enroll in the same course twice
        // Enforced at DATABASE level — even bypassing application logic can't create
duplicates
    }
)
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "enrollment_id")
    private Long enrollmentId;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "student_id", nullable = false)
    private Student student;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "course_id", nullable = false)
    private Course course;

    @Column(name = "enrollment_date", nullable = false, updatable = false)
    private LocalDateTime enrollmentDate;

    @Column(nullable = false, length = 20)
    private String status;            // "ENROLLED" | "COMPLETED" | "DROPPED"

    @PrePersist
    protected void onCreate() {
        enrollmentDate = LocalDateTime.now();
        if (status == null) status = "ENROLLED"; // Default on creation
    }
}
```

4.6 Database Schema (SQL DDL)

```
CREATE TABLE users (
    id        BIGINT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email     VARCHAR(100) UNIQUE NOT NULL,
    password  VARCHAR(255) NOT NULL,  -- BCrypt hash: always 60 chars
    role      VARCHAR(20) NOT NULL    -- "ROLE_ADMIN" | "ROLE_STUDENT" |
"ROLE_SUPER_ADMIN"
);

CREATE TABLE students (
    student_id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id      BIGINT NOT NULL,
    institute_id BIGINT,              -- NULL if institute deleted (ON DELETE SET NULL)
    phone        VARCHAR(15),
    department   VARCHAR(100),
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (institute_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE courses (
    course_id       BIGINT AUTO_INCREMENT PRIMARY KEY,
    course_name     VARCHAR(150) NOT NULL,
    description     TEXT,             -- Unlimited text (no length cap)
    duration        VARCHAR(50),
    instructor_name VARCHAR(100),
    content         TEXT,
    institute_id    BIGINT,
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (institute_id) REFERENCES users(id) ON DELETE SET NULL
);

CREATE TABLE enrollments (
    enrollment_id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    student_id      BIGINT NOT NULL,
    course_id       BIGINT NOT NULL,
    enrollment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    status          VARCHAR(20) DEFAULT 'ENROLLED',
    FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE,
    FOREIGN KEY (course_id)  REFERENCES courses(course_id)   ON DELETE CASCADE,
    UNIQUE KEY unique_student_course (student_id, course_id)  -- prevent duplicates
);
```

i INFO

ON DELETE CASCADE: Deleting a student automatically deletes all their enrollment rows (database enforces it, no application code needed). ON DELETE SET NULL: Deleting an institute sets institute_id to NULL in students/courses rows — the records survive but lose the institute link.

Chapter 5: Security Architecture — JWT & Spring Security

5.1 What is JWT?

JWT (RFC 7519) is a compact, self-contained token for securely transmitting claims. It has three Base64URL-encoded parts separated by dots: Header.Payload.Signature

```
Header (Base64URL of JSON):  { "alg": "HS256", "typ": "JWT" }
Payload (Base64URL of JSON): { "sub": "user@example.com", "iat": 1700000000, "exp":
1700086400 }
Signature:  HMAC_SHA256( base64(header) + "." + base64(payload), SECRET_KEY )

Full token: eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyQGV4YW1wbGUu....SflKxwRJSMeK...

NOTE: The payload is ENCODED (base64), NOT ENCRYPTED.
Anyone can decode and read it — NEVER put sensitive data (passwords) in JWT claims.
The SIGNATURE guarantees tamper-evidence: modifying the payload invalidates the
signature.
```

i INFO

Security: Only the server knows the SECRET_KEY. If an attacker modifies the payload (changes "sub" email or "exp" date), recomputing HMAC_SHA256 without the secret produces a different signature — the server detects tampering and rejects the token. Clients can READ the payload but cannot FORGE a valid signature.

5.2 JwtTokenProvider — Token Lifecycle

```
@Component
public class JwtTokenProvider {
    private final Key key;
    private final long jwtExpirationInMs;

    public JwtTokenProvider(
        @Value("${studenthub.jwt.secret}") String jwtSecret,     // from
application.properties
        @Value("${studenthub.jwt.expiration-ms}") long expMs) {  // 86400000 = 24 hours
        this.key = Keys.hmacShaKeyFor(jwtSecret.getBytes());      // String !' HMAC Key
object
        this.jwtExpirationInMs = expMs;
    }

    // Called once on successful login
    public String generateToken(Authentication auth) {
        String email = ((UserDetails) auth.getPrincipal()).getUsername();
        Date now = new Date();
        return Jwts.builder()
            .setSubject(email)                                  // "sub" = email address
            .setIssuedAt(now)                                   // "iat" = now
            .setExpiration(new Date(now.getTime() + jwtExpirationInMs)) // "exp" = +24h
            .signWith(key, SignatureAlgorithm.HS256)            // Sign with HMAC-SHA256
            .compact();                                         // Build JWT string
    }

    // Called on every authenticated request
    public String getEmailFromJWT(String token) {
        return Jwts.parserBuilder()
            .setSigningKey(key)         // Must match the signing key
            .build()
            .parseClaimsJws(token)      // Validates signature + expiry simultaneously
            .getBody().getSubject();    // Extract "sub" claim = email
    }

    public boolean validateToken(String token) {
        try {
            Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJws(token);
            return true; // Signature valid + not expired
        } catch (MalformedJwtException | ExpiredJwtException |
                 UnsupportedJwtException | IllegalArgumentException ex) {
            return false; // Any validation failure = invalid
        }
    }
}
```

5.3 JwtAuthenticationFilter

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    // OncePerRequestFilter: runs exactly once per request (prevents double-execution
    // in async dispatch or request forwarding scenarios)

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain) throws ... {
        String jwt = getJwtFromRequest(req);  // Extract from Authorization header

        if (StringUtils.hasText(jwt) && tokenProvider.validateToken(jwt)) {
            String email = tokenProvider.getEmailFromJWT(jwt);

            // Load FRESH user from DB on every request
            // Ensures role changes take effect immediately (no stale JWT data)
            UserDetails userDetails =
customUserDetailsService.loadUserByUsername(email);

            UsernamePasswordAuthenticationToken auth =
                new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());

            // ThreadLocal storage: auth available throughout this request's thread
            SecurityContextHolder.getContext().setAuthentication(auth);
        }

        chain.doFilter(req, res); // Always continue — authentication failure =
anonymous access
    }

    private String getJwtFromRequest(HttpServletRequest req) {
        String bearer = req.getHeader("Authorization");
        // Expected format: "Bearer eyJhbGciOiJIUzI1NiJ9..."
        if (StringUtils.hasText(bearer) && bearer.startsWith("Bearer ")) {
            return bearer.substring(7); // Remove 7-char "Bearer " prefix
        }
        return null;
    }
}
```

5.4 SecurityConfig

```
@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            // CORS: permits React dev server (port 5173) to call Spring API (port 8080)
            // Without CORS headers, browsers block cross-origin requests (Same-Origin
Policy)

            .csrf(AbstractHttpConfigurer::disable)
            // CSRF disabled: JWT is stateless, no cookies !’ CSRF attacks don't apply

            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            // No HTTP sessions — every request independently authenticated via JWT

            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()           // login/
register: public
                .requestMatchers("/api/super-admin/**").hasRole("SUPER_ADMIN")
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/courses/**").authenticated()    // any logged-in
user
                .requestMatchers("/api/enrollments/**").authenticated()
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtAuthenticationFilter,
                UsernamePasswordAuthenticationFilter.class) // JWT filter first
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
        // BCrypt: slow (intentional — costs ~100ms), auto-salted per hash,
        // adjustable work factor. Defeats brute-force and rainbow table attacks.
    }

    @Bean
    public CorsConfigurationSource corsConfigurationSource() {
        CorsConfiguration config = new CorsConfiguration();
        config.setAllowedOrigins(Arrays.asList(
            "http://localhost:5173",   // Vite dev server
            "http://localhost:3000",   // CRA dev server (if used)
            "http://localhost:8080"    // Same origin
        ));

config.setAllowedMethods(Arrays.asList("GET","POST","PUT","DELETE","OPTIONS","PATCH"));
        config.setAllowedHeaders(Arrays.asList("Authorization","Content-Type","Cache-
Control"));
        config.setAllowCredentials(true);
        UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
        source.registerCorsConfiguration("/**", config);
        return source;
    }
}
```

i INFO

hasRole("ADMIN") automatically checks for GrantedAuthority "ROLE_ADMIN" (Spring prepends "ROLE_"). In CustomUserDetailsService, we store the full name user.getRole().name() = "ROLE_ADMIN" as the authority, making both hasRole("ADMIN") and hasAuthority("ROLE_ADMIN") work correctly.

Chapter 6: REST API Reference

6.1 AuthController — /api/auth/**

Method	Endpoint	Auth	Response
POST	/api/auth/register	None	201 "Account registered successfully!"
POST	/api/auth/login	None	200 AuthResponse {token, role, studentId,...}
GET	/api/auth/me	Any	200 User profile data
GET	/api/auth/institutes	None	200 List<InstituteDto> for dropdown

Login Request/Response

```
// POST /api/auth/login
// Request: { "email": "john@example.com", "password": "secure123" }
// Response 200:
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",  // Store in localStorage, send in every request
header
  "id": 12,
  "email": "john@example.com",
  "fullName": "John Doe",
  "role": "ROLE_STUDENT",              // Frontend uses this for routing and UI
decisions
  "studentId": 7,                       // Only set if role = ROLE_STUDENT
  "instituteId": 5,                     // Institute's user ID
  "instituteName": "Tech Academy"       // For display
}
```

Register Request

```
// Student: POST /api/auth/register
{
  "accountType": "STUDENT",
  "fullName": "John Doe",
  "email": "john@example.com",
  "password": "secure123",          // @Size(min=6) validation
  "phone": "9876543210",            // @Pattern(^$|^d{10}$) — optional, 10 digits
  "department": "Computer Science",
  "instituteId": 5                  // Required for students
}

// Institute: POST /api/auth/register
{
  "accountType": "INSTITUTE",
  "fullName": "Tech Academy",
  "email": "academy@example.com",
  "password": "admin123"
}
```

6.2 CourseController — /api/courses/**

Method	Endpoint	Auth	Description
GET	/api/courses?page=0&size=10&search=java	Authenticated	Paginated course list with optional search
GET	/api/courses/{id}	Authenticated	Single course details
POST	/api/courses	ROLE_ADMIN	Create course for authenticated institute
PUT	/api/courses/{id}	ROLE_ADMIN	Update course (must own course)
DELETE	/api/courses/{id}	ROLE_ADMIN	Delete course + cascade delete enrollments

6.3 EnrollmentController — /api/enrollments/**

Method	Endpoint	Auth	Notes
POST	/api/enrollments?studentId=X&courseId=Y	ROLE_STUDENT	Can only enroll yourself in own institute's courses
GET	/api/enrollments/student/{id}	Student (self) / Admin	Students see own; Admins see institute's students
GET	/api/enrollments	ROLE_ADMIN	All enrollments for the authenticated institute
PUT	/api/enrollments/{id}/status	Student / Admin	Status: ENROLLED | COMPLETED | DROPPED
DELETE	/api/enrollments/{id}	Student / Admin	Cancel/withdraw enrollment

6.4 SuperAdmin Controller — /api/super-admin/**

All require ROLE_SUPER_ADMIN. Full platform management including per-institute data scoped via /institutes/{instId}/... URL pattern.

Method	Endpoint	Description
GET	/api/super-admin/stats	Platform totals: institutes, students, courses, enrollments
GET/POST	/api/super-admin/institutes	List all / Create new institute
PUT/DELETE	/api/super-admin/institutes/{id}	Update / Delete (cascades all data)
GET/POST	/api/super-admin/institutes/{id}/courses	List or create courses for an institute
PUT/DELETE	/api/super-admin/institutes/{id}/courses/{cid}	Update or delete specific course
GET/DELETE	/api/super-admin/institutes/{id}/students/{sid}	List or delete students
GET/PUT/DELETE	/api/super-admin/institutes/{id}/enrollments/{eid}	Manage enrollments
PUT	/api/super-admin/profile	Update super admin own profile

Chapter 7: Service Layer — Business Logic

7.1 UserService — Registration

```
@Transactional
public Student registerStudent(RegisterRequest request) {
    // 1. Validate institute exists and is ROLE_ADMIN
    User institute = userRepository.findById(request.getInstituteId())
        .filter(u -> u.getRole() == UserRole.ROLE_ADMIN)
        .orElseThrow(() -> new ResourceNotFoundException("Institute not found"));

    // 2. Create User account (authentication record)
    User user = User.builder()
        .email(request.getEmail())
        .password(passwordEncoder.encode(request.getPassword())) // BCrypt hash stored
        .role(UserRole.ROLE_STUDENT)
        .build();
    user = userRepository.save(user);  // INSERT into users table

    // 3. Create Student profile (linked to User + Institute)
    Student student = Student.builder()
        .user(user)            // OneToOne link
        .institute(institute)  // ManyToOne link
        .department(request.getDepartment())
        .build();
    return studentRepository.save(student); // INSERT into students table
    // @Transactional: if save(student) fails, save(user) is rolled back automatically
}
```

7.2 UserService — Cascading Delete

When deleteInstitute(instituteId) is called, four cleanup operations happen atomically:

```
@Transactional
public void deleteInstitute(Long instituteId) {
    User institute = userRepository.findById(instituteId)
        .filter(u -> u.getRole() == UserRole.ROLE_ADMIN)
        .orElseThrow(() -> new ResourceNotFoundException("Institute not found"));

    // Step 1: Delete student enrollments (must before deleting students)
    List<Student> students = studentRepository.findByInstitute(institute);
    for (Student student : students) {
        enrollmentRepository.deleteByStudentStudentId(student.getStudentId());
    }

    // Step 2: Delete students (CascadeType.ALL also deletes their User accounts)
    studentRepository.deleteAll(students);

    // Step 3: Delete course enrollments, then delete courses
    List<Course> courses = courseRepository.findByInstitute(institute);
    for (Course course : courses) {
        enrollmentRepository.deleteByCourseCourseId(course.getCourseId());
        courseRepository.delete(course);
    }

    // Step 4: Delete the institute User record itself
    userRepository.delete(institute);
    // If any step throws, @Transactional rolls back all previous steps !’ no partial
state
}
```

i INFO

Atomicity in Action: All four steps execute in one @Transactional method. If step 3 fails (e.g., database down), steps 1 and 2 are rolled back automatically. The database is always left in a consistent state — no orphaned enrollment records without parent students.

7.3 CourseService — Institute Scoping

```
// Security enforcement: resolve who the "current institute" is
private User getCurrentInstitute() {
    User user = userService.getCurrentlyAuthenticatedUser();

    if (user.getRole() == UserRole.ROLE_ADMIN) {
        return user; // Admin IS the institute — return themselves
    }

    // For students: find their linked institute
    Student student = studentRepository.findByUser(user)
        .orElseThrow(() -> new ResourceNotFoundException("Student profile not found"));
    return student.getInstitute();
    // Result: students can only see their own institute's courses
}

// Called before every course read/update/delete
private void ensureCourseBelongsToCurrentInstitute(Course course) {
    User institute = getCurrentInstitute();
    if (course.getInstitute() == null ||
        !course.getInstitute().getId().equals(institute.getId())) {
        // Treat as "not found" — no information leakage about other institutes' courses
        throw new ResourceNotFoundException("Course not found: " +
course.getCourseId());
    }
}
```

7.4 EnrollmentService — Business Rules

```
@Transactional
public EnrollmentDto enrollStudent(Long studentId, Long courseId) {
    User currentUser = userService.getCurrentlyAuthenticatedUser();
    Student student = studentRepository.findById(studentId)...;
    Course course = courseRepository.findById(courseId)...;

    // Rule 1: Only ROLE_STUDENT can enroll AND only themselves (not another student)
    if (currentUser.getRole() != UserRole.ROLE_STUDENT ||
        !student.getUser().getId().equals(currentUser.getId())) {
        throw new BadRequestException("Students can only enroll themselves");
    }

    // Rule 2: Cross-institute enrollment forbidden
    if (!student.getInstitute().getId().equals(course.getInstitute().getId())) {
        throw new BadRequestException("Can only enroll in your institute's courses");
    }

    // Rule 3: Duplicate prevention (also enforced by DB UNIQUE constraint as backup)
    if (enrollmentRepository.existsByStudentStudentIdAndCourseCourseId(studentId,
courseId)) {
        throw new BadRequestException("Already enrolled in this course!");
    }

    Enrollment enrollment = Enrollment.builder()
        .student(student).course(course).status("ENROLLED").build();
    return convertToDto(enrollmentRepository.save(enrollment));
    // @PrePersist sets enrollmentDate = now() and status = "ENROLLED" automatically
}
```

Chapter 8: Repository Layer — Data Access

Spring Data JPA repositories are Java interfaces. Spring auto-generates SQL implementations at application startup by parsing method names. JpaRepository<Entity, ID> provides 18+ built-in operations including save(), findById(), findAll(), delete(), count().

8.1 UserRepository

```
public interface UserRepository extends JpaRepository<User, Long> {
    // SELECT * FROM users WHERE email = ?  !’  Optional (may be absent)
    Optional<User> findByEmail(String email);

    // SELECT COUNT(*) > 0 FROM users WHERE email = ?  !’  boolean
    boolean existsByEmail(String email);

    // SELECT * FROM users WHERE role = ? ORDER BY full_name ASC
    List<User> findByRoleOrderByFullNameAsc(UserRole role);
}
```

8.2 CourseRepository

```
public interface CourseRepository extends JpaRepository<Course, Long> {

    // SELECT * FROM courses WHERE institute_id = ?
    List<Course> findByInstitute(User institute);

    // Same + LIMIT ? OFFSET ? for pagination
    Page<Course> findByInstitute(User institute, Pageable pageable);

    // Complex 3-field search scoped to institute:
    // WHERE (inst_id=? AND LOWER(course_name) LIKE ?)
    //    OR (inst_id=? AND LOWER(instructor_name) LIKE ?)
    //    OR (inst_id=? AND LOWER(description) LIKE ?)
    Page<Course> findByInstituteAndCourseNameContainingIgnoreCaseOrInstituteAndInstructo
rNameContainingIgnoreCaseOrInstituteAndDescriptionContainingIgnoreCase(
        User i1, String name, User i2, String instr, User i3, String desc, Pageable
pageable);

    long countByInstitute(User institute); // SELECT COUNT(*) WHERE institute_id = ?
}
```

8.3 EnrollmentRepository

```
public interface EnrollmentRepository extends JpaRepository<Enrollment, Long> {

    List<Enrollment> findByStudentStudentId(Long studentId);    // Student's enrollments
    List<Enrollment> findByCourseInstituteId(Long instituteId); // Institute's
enrollments

    // SELECT COUNT(*) > 0 WHERE student_id = ? AND course_id = ?
    boolean existsByStudentStudentIdAndCourseCourseId(Long studentId, Long courseId);

    long countByCourseInstituteId(Long instituteId);
    long countByCourseInstituteIdAndStatus(Long instituteId, String status);

    // DELETE FROM enrollments WHERE student_id = ?  (bulk delete on student removal)
    void deleteByStudentStudentId(Long studentId);
    void deleteByCourseCourseId(Long courseId);
}
```

Chapter 9: Exception Handling

@RestControllerAdvice intercepts ALL exceptions from ALL controllers and transforms them into consistent JSON responses. Clients always receive the same structured error format regardless of which layer threw the exception.

9.1 Exception Classes

```
// HTTP 404 Not Found — entity does not exist
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String message) { super(message); }
}
// Example: "Course not found with ID: 99"

// HTTP 400 Bad Request — business rule violation
public class BadRequestException extends RuntimeException {
    public BadRequestException(String message) { super(message); }
}
// Example: "Email is already registered!"

// Why RuntimeException (unchecked)?
// 1. No 'throws' declaration needed in method signatures (cleaner code)
// 2. @Transactional automatically rolls back on RuntimeException subclasses
// 3. GlobalExceptionHandler catches globally without try-catch in every method
```

9.2 GlobalExceptionHandler

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<Map<String,Object>> handleNotFound(ResourceNotFoundException
ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(Map.of(
            "timestamp", LocalDateTime.now(), "status", 404,
            "error", "Not Found", "message", ex.getMessage()
        ));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)  // from @Valid
    public ResponseEntity<Map<String,Object>>
handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> fieldErrors = new HashMap<>();
        ex.getBindingResult().getAllErrors().forEach(e -> {
            fieldErrors.put(((FieldError) e).getField(), e.getDefaultMessage());
        });
        // Returns: {"status":400, "validationErrors": {"email":"Invalid email format"}}
        return ResponseEntity.badRequest().body(Map.of(
            "status", 400, "error", "Validation Error", "validationErrors",
fieldErrors));
    }

    @ExceptionHandler(BadCredentialsException.class)  // wrong password on login
    public ResponseEntity<Map<String,Object>> handleBadCreds(BadCredentialsException
ex) {
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(Map.of(
            "status", 401, "message", "Invalid email or password"));
    }

    @ExceptionHandler(Exception.class)  // catch-all for unexpected errors
    public ResponseEntity<Map<String,Object>> handleGeneral(Exception ex) {
        return ResponseEntity.internalServerError().body(Map.of(
            "status", 500, "message", ex.getMessage()));
    }
}
```

9.3 Error Response Summary

Exception	HTTP Code	Trigger Scenario
ResourceNotFoundException	404 Not Found	Entity ID not in database
BadRequestException	400 Bad Request	Business rule violation (duplicate, missing field)
MethodArgumentNotValidException	400 Bad Request	@Valid annotation fails on DTO
BadCredentialsException	401 Unauthorized	Wrong password during login
Exception (catch-all)	500 Server Error	Unexpected runtime exceptions

Chapter 10: Frontend Architecture — React SPA

10.1 Application Structure

```
// main.jsx — Absolute entry point
ReactDOM.createRoot(document.getElementById('root')).render(<App />);

// App.jsx — Router + all routes + global providers
function App() {
  return (
    <Router>
      <AuthProvider>          // Global auth state wraps entire tree
        <Navbar />            // Role-aware navigation (always rendered)
        <main>
          <Routes>
            <Route path="/"        element={<Home />} />           // Public
            <Route path="/login"   element={<Login />} />           // Public
            <Route path="/register" element={<Register />} />       // Public

            // Protected — requires ROLE_SUPER_ADMIN
            <Route path="/super-admin" element={
              <ProtectedRoute allowedRoles={['ROLE_SUPER_ADMIN']}>
                <SuperAdminDashboard />
              </ProtectedRoute>
            } />

            // Protected — requires ROLE_ADMIN
            <Route path="/admin" element={
              <ProtectedRoute allowedRoles={['ROLE_ADMIN']}>
                <AdminDashboard />
              </ProtectedRoute>
            } />

            // Protected — requires ROLE_STUDENT
            <Route path="/student" element={
              <ProtectedRoute allowedRoles={['ROLE_STUDENT']}>
                <StudentDashboard />
              </ProtectedRoute>
            } />

            <Route path="*" element={<Navigate to="/" replace />} /> // Catch-all
          </Routes>
        </main>
        <Footer />
      </AuthProvider>
    </Router>
  );
}
```

10.2 AuthContext — Global State

```
const AuthContext = createContext(null);

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);       // null = not logged in
  const [loading, setLoading] = useState(true); // checking localStorage on mount

  useEffect(() => {
    // Restore session from localStorage on page refresh
    const storedUser = localStorage.getItem('user');
    const token = localStorage.getItem('token');
    if (storedUser && token) setUser(JSON.parse(storedUser));
    setLoading(false);
  }, []); // [] = run once after mount

  const login = async (email, password) => {
    const { data } = await API.post('/auth/login', { email, password });
    const { token, ...userData } = data;          // Separate token from user info
    localStorage.setItem('token', token);          // Persist JWT across refreshes
    localStorage.setItem('user', JSON.stringify(userData));
    setUser(userData);                             // React re-renders all consumers
    return userData;
  };

  const logout = () => {
    localStorage.removeItem('token'); localStorage.removeItem('user');
    setUser(null); // ProtectedRoute detects null user !' redirects to /login
  };

  return (
    <AuthContext.Provider value={{
      user, login, logout, register,
      isAdmin:      () => user?.role === 'ROLE_ADMIN',
      isStudent:    () => user?.role === 'ROLE_STUDENT',
      isSuperAdmin: () => user?.role === 'ROLE_SUPER_ADMIN',
    }}>
      {children}
    </AuthContext.Provider>
  );
};

export const useAuth = () => useContext(AuthContext);  // Custom hook for clean API
```

10.3 Axios Interceptors

```
const API = axios.create({ baseURL: 'http://localhost:8080/api' });

// REQUEST INTERCEPTOR — runs before EVERY API call
API.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) config.headers['Authorization'] = `Bearer ${token}`;
  // Every API call now has: Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
  return config;
});

// RESPONSE INTERCEPTOR — runs on EVERY response
API.interceptors.response.use(
  response => response,  // 2xx: pass through unchanged
  error => {
    if (error.response?.status === 401) {
      // JWT expired or invalid — clear everything, send to login
      localStorage.removeItem('token'); localStorage.removeItem('user');
      if (window.location.pathname !== '/login')
        window.location.href = '/login?expired=true';
    }
    return Promise.reject(error); // Re-throw for local .catch() handlers
  }
);
```

i INFO

Interceptors eliminate repetitive code. Without them, every component making API calls would need to manually add the Authorization header and handle 401 errors. With interceptors, all components use simple API.get(url) calls — JWT attachment and expiry handling happen transparently.

10.4 ProtectedRoute

```
const ProtectedRoute = ({ children, allowedRoles }) => {
  const { user, loading } = useAuth();

  if (loading) return <div>Loading...</div>;    // Wait for localStorage restoration

  if (!user) return <Navigate to="/login" replace />;   // Not authenticated

  if (allowedRoles && !allowedRoles.includes(user.role)) {
    return <Navigate to="/" replace />;         // Wrong role — graceful denial
  }

  return children; // Access granted
};
```

Chapter 11: Configuration, Build & Deployment

11.1 application.properties

```
# DATABASE
spring.datasource.url=jdbc:mysql://localhost:3306/studenthub_db
  ?createDatabaseIfNotExist=true&useSSL=false&serverTimezone=UTC&allowPublicKeyRetrieval
=true
spring.datasource.username=root
spring.datasource.password=cdac
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / HIBERNATE
spring.jpa.hibernate.ddl-auto=update     # Creates tables on first run; alters on
subsequent
spring.jpa.show-sql=true                  # Log all SQL to console (debug)
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
spring.jpa.open-in-view=false            # Prevents lazy-loading antipattern

# JWT
studenthub.jwt.secret=404E635266556A586E32723575387...  # 64-char hex = 32-byte HMAC key
studenthub.jwt.expiration-ms=86400000                    # 24 hours

# FILE UPLOAD (optional)
spring.servlet.multipart.max-file-size=5MB
spring.servlet.multipart.max-request-size=5MB
```

11.2 Running Backend & Frontend

```
# % %  BACKEND % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
# From StudentHub/backend/
..mavenapache-maven-3.9.6†–æ×fâæ6ÖB 7 &–ærÖ&ö÷C§'Và

# Startup order:
# 1. Spring Boot scans classpath !' registers all beans
# 2. Hibernate connects to MySQL !' runs CREATE/ALTER TABLE DDL
# 3. CommandLineRunner seeds superadmin@studenthub.com if not exists
# 4. Embedded Tomcat starts !' http://localhost:8080 is ready

# % %  FRONTEND % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
# From StudentHub/frontend/
npm install        # Install dependencies to node_modules/
npm run dev        # Vite dev server starts at http://localhost:5173
# Vite hot-reloads on file save — no manual restart needed
```

11.3 Database Seeding

```
// In StudentHubApplication.java
@Bean
public CommandLineRunner initDatabase(UserRepository repo, PasswordEncoder encoder) {
    return args -> {
        // existsByEmail check prevents duplicate on every application restart
        if (!repo.existsByEmail("superadmin@studenthub.com")) {
            repo.save(User.builder()
                .fullName("Super Admin")
                .email("superadmin@studenthub.com")
                .password(encoder.encode("superadmin123")) // BCrypt hash stored in DB
                .role(UserRole.ROLE_SUPER_ADMIN)
                .build());
            System.out.println("Seeded: superadmin@studenthub.com / superadmin123");
        }
    };
}
```

! NOTE

Change the superadmin password immediately in production! The CommandLineRunner only seeds the account if it does not exist. Update via PUT /api/super-admin/profile — the new BCrypt hash replaces the old one in the database. The seed password "superadmin123" is public knowledge from this documentation.

Chapter 12: Roles, Responsibilities & Access Control

12.1 Three-Tier RBAC Model

Role	Scope	Created By	Key Capability
ROLE_SUPER_ADMIN	Platform-wide	Auto-seeded at startup	Manage all institutes, view all data
ROLE_ADMIN	Own institute only	Self-registration or Super Admin	Manage own courses/students
ROLE_STUDENT	Own institute only	Self-registration (needs institute)	Enroll in own institute courses

12.2 Access Control Matrix

Feature	SUPER_ADMIN	ADMIN	STUDENT
Platform stats	Yes (all)	No	No
Manage institutes (CRUD)	Yes (all)	No	No
View/Edit own profile	Yes	Yes	Yes
View courses	Yes (any inst)	Yes (own)	Yes (own inst)
Create/Edit/Delete courses	Yes (any)	Yes (own)	No
View students	Yes (all)	Yes (own inst)	No
Delete student	Yes (any)	Yes (own inst)	No
Enroll in course	No	No	Yes (same inst)
View enrollments	Yes (all)	Yes (own inst)	Yes (own only)
Update/Cancel enrollment	Yes (any)	Yes (own inst)	Yes (own only)
Dashboard stats	Yes (platform)	Yes (scoped)	No

Chapter 13: Key Concepts Glossary

13.1 Core Java Concepts

Annotations (@interface)

@Entity, @RestController, @Transactional are metadata tags. They don't change Java execution but signal frameworks how to process annotated elements. Spring reads annotations at startup via reflection to discover and configure beans, map controllers, and manage transactions — replacing verbose XML configuration.

Optional<T> — Null Safety

Optional<T> is a container that may or may not hold a value. userRepository.findByEmail(email).orElseThrow(() -> new ResourceNotFoundException(...)) either returns the User or throws an exception. NullPointerException is impossible because Optional forces explicit handling of the absent case. Optional eliminates the #1 Java runtime error.

Generics — Type Safety

List<CourseDto> is a list that can only contain CourseDto objects — the compiler enforces this. Page<Course> wraps pagination data. ResponseEntity<T> makes the HTTP response body type explicit. Generics catch type mismatches at compile time instead of runtime ClassCastException.

Stream API — Functional Processing

repository.findAll().stream().filter(s -> s.getDepartment().equals("CS")).map(this::convertToDto).collect(Collectors.toList()) processes the list functionally. Each operation (filter, map, collect) is composable. Streams are lazy — no work happens until the terminal operation (collect). More readable and concise than equivalent for-loops.

13.2 Spring Framework

@Autowired — Dependency Injection

Spring's IoC (Inversion of Control) container manages all bean creation and wiring. @Autowired tells Spring to inject the required bean. Instead of new UserService(new UserRepository(...)), Spring creates one singleton instance of each bean and injects it everywhere needed. This decouples classes and makes unit testing easy (inject mock beans).

@Transactional — ACID Guarantees

Wraps a method in a database transaction via AOP proxy. If the method completes normally !' COMMIT. If a RuntimeException is thrown !' ROLLBACK. Propagation.REQUIRED (default): nested @Transactional methods join the existing transaction. If the outer method fails, all inner operations roll back too — guaranteed consistency for multi-step operations.

ResponseEntity<T>

Full control over the HTTP response: ResponseEntity.ok(dto) !' 200 OK, new ResponseEntity<>(dto, HttpStatus.CREATED) !' 201, ResponseEntity.noContent().build() !' 204, ResponseEntity.status(404).body(error) !' 404. Without ResponseEntity, return value becomes the body with 200 status.

13.3 Security Concepts

BCrypt — Password Hashing

BCrypt hash format: $2a$10$SALT...HASH. $2a$=version, 10=work factor (2^10 iterations), 22-char salt, 31-char hash. The salt is embedded — BCryptPasswordEncoder.matches() extracts it for comparison. Same password !' different hash each time (salted). Defeating brute force: ~100ms per hash attempt on modern hardware.

SecurityContextHolder — Thread-Local Auth

SecurityContextHolder uses ThreadLocal<SecurityContext> — each HTTP request thread has its own isolated context. After JwtAuthenticationFilter populates it, any code on that thread can call SecurityContextHolder.getContext().getAuthentication() to get the current user. No need to pass user as method parameter through every call in the chain.

CORS — Cross-Origin Resource Sharing

Same-Origin Policy: browsers block JavaScript from fetching a different origin (protocol+domain+port). React (port 5173) calling Spring (port 8080) = cross-origin. CORS headers from the server tell the browser it's permitted: Access-Control-Allow-Origin: http://localhost:5173. Without CORS config, React API calls fail with network errors.

13.4 Database Concepts

N+1 Query Problem

Loading 100 courses then accessing course.getInstitute() in a loop with LAZY fetch !' 101 queries (1 for courses + 100 for each institute). StudentHub uses FetchType.EAGER everywhere — Hibernate uses SQL JOINs to fetch institutes in the same query. Trade-off: slightly more data per query vs. dramatically fewer queries.

Database Indexing

MySQL auto-creates B-tree indexes for PRIMARY KEY and UNIQUE KEY columns. email UNIQUE !' O(log n) lookup vs O(n) table scan. UNIQUE KEY on (student_id, course_id) in enrollments creates a composite index for fast duplicate checking. Indexes significantly speed up reads at the cost of slightly slower writes (index maintenance).

Foreign Keys & Referential Integrity

Foreign keys prevent orphaned records. ON DELETE CASCADE: deleting a student auto-deletes their enrollment rows. ON DELETE SET NULL: deleting an institute sets institute_id = NULL in students/ courses (records survive, lose the link). These constraints are enforced by MySQL — even direct SQL manipulation respects them.

13.5 Frontend Concepts

React Hooks

Hooks give functional components access to state and lifecycle. useState(null) !' reactive variable + setter. useEffect(fn, []) !' runs once after mount (like componentDidMount). useEffect(fn, [dep]) !' re-runs when dep changes. useContext(AuthContext) !' consume context value. Custom hooks (useAuth) wrap useContext for cleaner component code.

Virtual DOM & Reconciliation

React maintains a Virtual DOM (JavaScript object tree). On state change, React renders the new Virtual DOM, diffs it against the previous version, and updates only the changed real DOM nodes. This is more efficient than re-rendering the full page. The key prop on list items helps React identify which items changed during list re-renders.

localStorage vs HttpOnly Cookies

localStorage: JavaScript-accessible, persistent. Vulnerable to XSS (malicious scripts can read it). HttpOnly cookies: invisible to JavaScript (XSS-resistant), auto-sent with requests. StudentHub uses localStorage (simpler, common in SPAs). Production apps with high security requirements should prefer HttpOnly + SameSite=Strict cookies.

Chapter 14: End-to-End Flow Walkthroughs

14.1 User Login Flow

- 1. User types email + password in Login.jsx !’ submits form
- 2. AuthContext.login() called !’ API.post("/auth/login", credentials)
- 3. Request interceptor runs — no JWT yet (public endpoint), just Content-Type header
- 4. Tomcat receives request on port 8080
- 5. JwtAuthenticationFilter: no Authorization header !’ skips !’ passes to next filter
- 6. Security rules: /api/auth/login matches permitAll() !’ authorized without JWT
- 7. AuthController.login(@RequestBody AuthRequest) called
- 8. userService.authenticateUser() !’ authenticationManager.authenticate(...)
- 9. Spring calls CustomUserDetailsService.loadUserByUsername(email)
- 10. SELECT * FROM users WHERE email = ? !’ User loaded from MySQL
- 11. BCryptPasswordEncoder.matches(inputPwd, storedHash) !’ true or BadCredentialsException
- 12. JwtTokenProvider.generateToken() !’ builds JWT: subject=email, exp=+24h, signed HMAC-SHA256
- 13. User fetched again !’ AuthResponse built: {token, role, studentId, instituteName, ...}
- 14. ResponseEntity.ok(authResponse) !’ HTTP 200 + JSON body
- 15. React: localStorage.setItem("token", token) !’ setUser(userData) !’ state update
- 16. React Router: navigate("/admin") or "/student" based on user.role

14.2 Student Enrolls in a Course

- 1. StudentDashboard shows available courses !’ student clicks "Enroll"
- 2. API.post("/api/enrollments?studentId=7&courseId=12")
- 3. Request interceptor attaches: Authorization: Bearer eyJ... from localStorage
- 4. JwtAuthenticationFilter extracts token !’ validates HMAC signature + expiry
- 5. Email extracted from JWT sub claim !’ user loaded from DB
- 6. SecurityContext populated with [ROLE_STUDENT] authority
- 7. Security rule: /api/enrollments/** requires authenticated() !’ pass
- 8. EnrollmentController.enrollStudent(7, 12)
- 9. EnrollmentService: getCurrentlyAuthenticatedUser() !’ User(ROLE_STUDENT)
- 10. studentRepository.findById(7) !’ Student (user.id=12, institute.id=5)
- 11. courseRepository.findById(12) !’ Course (institute.id=5)
- 12. Validation: student.getUser().getId()==currentUser.getId() !’ 12==12 (self-enrollment) OK
- 13. Validation: student.getInstitute().getId()==course.getInstitute().getId() !’ 5==5 OK
- 14. existsByStudentStudentIdAndCourseCourseId(7, 12) !’ false (not enrolled yet) OK
- 15. Enrollment built !’ enrollmentRepository.save() !’ INSERT into enrollments
- 16. @PrePersist: enrollmentDate=now(), status="ENROLLED" auto-set
- 17. ResponseEntity(enrollmentDto, 201 CREATED)
- 18. React: updates enrollment list, shows success notification

14.3 Admin Deletes a Student

- 1. AdminDashboard: admin clicks Delete on student row
- 2. API.delete("/api/admin/students/42") with JWT in header
- 3. JWT filter validates !’ Admin user loaded !’ SecurityContext set [ROLE_ADMIN]
- 4. Security: /api/admin/** requires hasRole("ADMIN") !’ pass
- 5. AdminController.deleteStudentAccount(42) !’ studentService.deleteStudent(42)

- 6. studentRepository.findById(42) !’ Student with institute.id=5
- 7. ensureSameInstitute(): getCurrentInstituteUser().id==student.getInstitute().id !’ 5==5 OK
- 8. enrollmentRepository.deleteByStudentStudentId(42) !’ DELETE FROM enrollments WHERE student_id=42
- 9. studentRepository.delete(student) !’ DELETE FROM students WHERE student_id=42
- 10. CascadeType.ALL propagates !’ DELETE FROM users WHERE id=(student's user_id)
- 11. @Transactional: all deletes commit atomically !’ ResponseEntity.noContent() !’ 204
- 12. React: removes student from displayed table without page reload

Chapter 15: Summary & Best Practices

15.1 Architecture Strengths

- Clear Separation of Concerns: Each layer has one job — testable and maintainable independently
- Stateless JWT Auth: No server sessions !' horizontal scaling possible without sticky sessions
- Multi-Tenant Isolation: Data scoping enforced in service layer + URL-level @PreAuthorize
- DTO Pattern: API contract independent of DB schema, no accidental sensitive data exposure
- Centralized Error Handling: @RestControllerAdvice gives consistent JSON error format
- Declarative Security: Access rules visible at annotation level — easy to audit
- Transactional Integrity: @Transactional ensures ACID guarantees for complex multi-step operations
- Spring Data JPA: Zero SQL for standard operations; derived queries are type-safe and readable

15.2 Lombok Annotations

Annotation	Generated Code
@Data	getters + setters + equals() + hashCode() + toString()
@Builder	Entity.builder().field(val).build() builder pattern
@NoArgsConstructor	No-arg constructor (required by JPA specification)
@AllArgsConstructor	Constructor with all fields
@Slf4j	private static final Logger log = LoggerFactory.getLogger(...)

15.3 Default Credentials

Role	Email	Password	Source
ROLE_SUPER_ADMIN	superadmin@studenthub.com	superadmin123	CommandLineRunner seed
ROLE_ADMIN	Any email	Chosen at registration	/api/auth/register (INSTITUTE)
ROLE_STUDENT	Any email	Chosen at registration	/api/auth/register (STUDENT)

! NOTE

Change superadmin password immediately in production! Use PUT /api/super-admin/profile with a new password. BCrypt makes the old hash irreversible. Storing secrets in application.properties is acceptable for development but production should use environment variables, AWS Parameter Store, or HashiCorp Vault.

15.4 Production Improvement Roadmap

- HttpOnly + SameSite=Strict cookies for JWT (eliminates XSS token theft risk)
- Refresh Token mechanism (short-lived 15min access + long-lived 30-day refresh)
- Email verification on registration (prevents fake accounts, validates emails)
- Rate limiting on /api/auth/** with Bucket4j (prevent brute-force attacks)
- Spring Profiles (application-dev.properties, application-prod.properties)
- Externalize JWT secret to environment variables or secrets management service
- Springdoc OpenAPI 3.0 (interactive Swagger UI for API documentation)
- JUnit 5 + Mockito unit tests; Spring Boot Test integration tests
- Flyway/Liquibase for version-controlled database migrations (safer than ddl-auto=update)
- Docker + Docker Compose for reproducible deployment across environments

- Structured JSON logging with Logback for log aggregation (ELK Stack / Splunk)
- API versioning strategy (/api/v1/**) for backward compatibility